

Fiches de Cours

Master IMA

Université de Toulouse III - Paul Sabatier, Toulouse
Années universitaires 2025-2027

8 octobre 2025

Table des matières

I	Algorithmique	3
1	Flot maximal dans un graphe	4
1.1	Rappels sur les graphes	4
1.2	Cycle et flot	6
1.3	Flot et coupure	7
1.4	Flot maximal : principe de Ford-Fulkerson	9
2	Programmation Linéaire	11
2.1	Généralités	11
2.2	Algorithme du Simplexe et Dualité	15

Algorithmique

Chapitre 1

Flot maximal dans un graphe

Contents

1.1	Rappels sur les graphes	4
1.2	Cycle et flot	6
1.3	Flot et coupure	7
1.3.1	Flot maximal dans un réseau	7
1.3.2	Théorème de Coupure minimale	8
1.4	Flot maximal : principe de Ford-Fulkerson	9
1.4.1	Recherche de Flot Maximal	9
1.4.2	Recherche de flot maximal de coût minimal	10

L'objectif de ce chapitre est d'être capable d'organiser efficacement un flot d'objets dans un réseau. Par exemple, on souhaite amener le maximum de camions d'un point A à un point B en passant par certaines routes. De même, on souhaiterait savoir comment router efficacement le plus de paquets dans un réseau informatique.

Tout ces problèmes se généralisent en ce que l'on appelle la *recherche de flot maximal dans un graphe orienté*. Nous allons ici définir proprement ce genre de problème, donner des conditions d'existence de flot maximal et proposer des algorithmes pour les construire.

1.1 Rappels sur les graphes

Pour représenter une structure de réseau (routier, informatique, etc...), on utilise les graphes. Ce cours fait donc suite à celui sur la théorie des graphes enseigné en licence. Effectuons quelques rappels.

Définition (Graphe Orienté) . On appelle graphe orienté un couple d'ensembles finis $G = (X, E)$ où $X = \{1, \dots, n\}$ représente les sommets du graphe et $E = \{(x_i, y_j), \dots\}$, $x_i, y_j \in X$ l'ensemble ordonné des arrêtes du graphe. Pour un graphe orienté, les notions de graphe simple et multigraphes sont les mêmes que pour le cas non orienté.

Exemple (Graphes et leur représentation gravarphique) Soient G_1 et G_2 deux graphes, en voici une représentation :

Définition (Voisinage) . Le voisinage d'un sommet x de G est l'ensemble des sommets

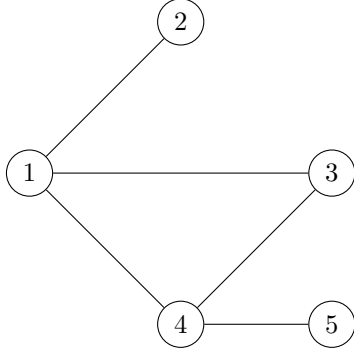


FIGURE 1.1 – Graphe non orienté

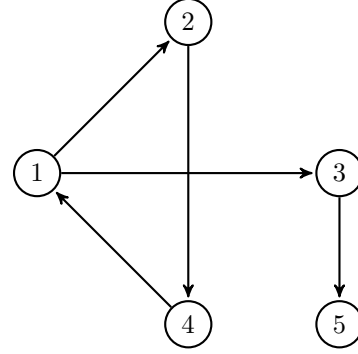


FIGURE 1.2 – Graphe orienté

y de G tels qu'il existe une arête entre x et y dans G . On le note $V(x)$.

Remarque Si x est dans le voisinage de y , on dira que x est adjacent à y et inversement.

Définition (Degré) . Le degré d'un sommet x de G est le cardinal du voisinage x . On le note $d(x)$. Un sommet de voisinage nul est dit *isolé* et un sommet de voisinage égal à 1 est dit *pendant*.

Définition (Voisinage entrant et sortant) . Soit G un graphe orienté et x un sommet de G . On définit deux types de voisinages :

- *Voisinage entrant* : noté $V^-(X)$ est l'ensemble des prédécesseurs de x .
- *Voisinage sortant* : noté $V^+(X)$ est l'ensemble des successeurs de x .

On définira comme précédemment le degré sortant et le degré entrant d'un sommet x .

Définition (Chaîne) . On appelle chaîne de $G = (X, E)$ de longueur n toute suite alternée de sommets et d'arrêtes de G telle que :

$$c = (x_0, a_1, x_1, \dots, a_n, x_n), \text{ telle que } \forall i \in \llbracket 1, n \rrbracket, a_i = (x_{i-1}, x_i)$$

Ici, n représente le nombre d'arrêtes de la chaîne. Dans le cas d'un graphe simple, on notera les chaînes de la façon suivante :

$$c = (x_0, \dots, x_n) \quad \text{ou} \quad c = x_0 - x_1 - \dots - x_n$$

Dans le cas des graphes orientés, on notera une chaîne simplement avec les arrêtes dont elle est constituée :

$$c = (u_1, \dots, u_n), \forall i \in \llbracket 1, n \rrbracket u_i \in E$$

Définition (Accessibilité) . Soit $G = (X, E)$ un graphe. On a :

- Soient x et y deux sommets de G . On dit que y est accessible à partir de x s'il existe une chaîne joignant x et y dans G .
- G est dit *connexe* ssi $\forall x \in X, \forall y \in X, y$ est accessible à partir de x .

- L'accessibilité est une relation d'équivalence entre les sommets.
Ses classes d'équivalences sont les composantes connexes de G .

Définition (Chaîne Simple) . Une chaîne est *simple* si elle ne passe pas deux fois par la même *arrête* et elle est dite *élémentaire* si elle ne passe pas deux fois par le même *sommet*. On remarquera facilement qu'une chaîne élémentaire est simple.

Définition (Cycle) . Un cycle de G est une chaîne simple dont le départ et l'arrivée sont le même sommet. Un cycle est donc de la forme :

$$c = x_0 - x_1 - \dots - x_{n-1} - x_0$$

Théorème (Propriétés des cycles et chaînes) . Soit G un graphe à $n \in \mathbb{N}$ sommets.

- Toute chaîne élémentaire a une longueur inférieure à $n - 1$
- Toute cycle élémentaire a une longueur inférieure à n
- De toute chaîne, on peut en extraire une chaîne élémentaire.

1.2 Cycle et flot

Définissons maintenant la notion de flot dans un cas général.

Définition (Vecteur de chaîne) . Soit $c = (u_1, \dots, u_n)$ une *chaîne élémentaire* dans un graphe orienté $G = (X, E)$ tel que $|E| = n$ et d'arcs u_i . On appelle *vecteur de chaîne* associé à c l'application φ associée à c telle que :

$$\varphi : \begin{cases} E \longrightarrow \mathbb{Z} \\ u_i \longmapsto \begin{cases} 1 & \text{si } u_i \text{ est dans le sens de } c \\ -1 & \text{si } u_i \text{ n'est pas dans le sens de } c \\ 0 & \text{si } u_i \text{ n'est pas dans } c \end{cases} \end{cases}$$

On la note en général $\varphi = (\varphi(u_1), \varphi(u_2), \dots, \varphi(u_n)) \in \mathbb{Z}^n$.

Venons en maintenant à la définition phare de ce chapitre :

Définition (Flot) . Soit $G = (X, E, c)$ un graphe orienté. On appelle *flot* dans G tout vecteur de cycle φ de G qui vérifie la loi de Kirchhoff :

$$\forall x \in X, \quad \sum_{u_i \in \omega^-(\{x\})} \varphi(u_i) = \sum_{u_i \in \omega^+(\{x\})} \varphi(u_i)$$

Et on note $\varphi^i = \varphi(u_i)$ le *flot* de l'arrête u_i .

Remarque Un flot est donc un chaîne élémentaire d'un graphe orienté dont, pour chaque noeud, la somme des flots sortants est égale à la somme des flots entrants.

Propriété (Flots) . Soit $G = (X, E, c)$ un graphe tel que $|E| = n$, alors :

- Le vecteur nul de $\mathbb{Z}^n$ est un flot de G .
- Toute combinaison linéaire de flots de G est aussi un flot de G .
- Tout vecteur cyclique de G est un flot de G .

Exemple Soit G le graphe suivant : Le vecteur $\varphi_1 = (2, -2, 1, 3, 4, -1, 1)$ est un flot de G . Le

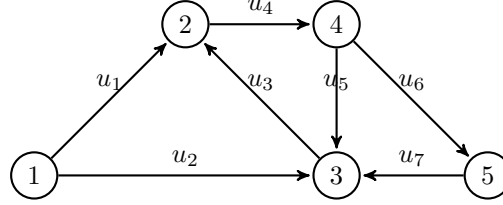


FIGURE 1.3 – flot dans un graphe

vecteur $\varphi_2 = (0, 0, 0, 0, 1, -1, 1)$ est aussi un flot. Soit $v = 2\varphi_1 + 3\varphi_2$, c'est aussi un flot de G .

Proposition (Loi de Kirchoff généralisée) Soit $G = (X, E, c)$ et $A \subset E$ non vide. Un chaîne φ est un flot de G ssi :

$$\sum_{u \in \omega^-(A)} \varphi(u) = \sum_{u \in \omega^+(A)} \varphi(u)$$

1.3 Flot et coupure

1.3.1 Flot maximal dans un réseau

Dans cette section, nous revenons à notre problème de départ en définissant la notion de graphe de transport. Graphes dans lesquels nous chercherons à construire des flots maximaux.

Définition (Réseau de Transport) . Un *réseau de transport* est un graphe orienté pondéré $R = (X, E, c)$ qui contient :

- un sommet sans prédécesseur appelé *source* (noté s).
- un sommet sans fils appelé *puits* (noté t).
- c , une application, qui à chaque arc, lui associe sa capacité notée c :

$$c : \begin{cases} E \longrightarrow \mathbb{R} \cup \{\infty\} \\ u = (x, y) \longmapsto c(u) \end{cases}$$

- un arc fictif de t à s doté d'une capacité infinie, appelé *arc retour*.

Définition (Flot compatible) . Un *flot compatible* sur un réseau $R = (X, E, c)$ tel que $|E| = n$ est un vecteur $\varphi \in \mathbb{Z}^n$ qui vérifie :

- (**Conservation de Kirchhoff**) pour tout $x \in X \setminus \{s, t\}$ (hors source s et puits t), on a :

$$\sum_{u \in \omega^+(x)} \varphi(u) = \sum_{u \in \omega^-(x)} \varphi(u)$$

où $\omega^+(x)$ (resp. $\omega^-(x)$) est l'ensemble des arcs sortants (resp. entrants) de x .

— (**Compatibilité avec les capacités**) :

$$\forall u \in E, \quad 0 \leq \varphi(u) \leq c(u).$$

La *valeur* du flot, notée $v(\varphi)$, est définie par :

$$v(\varphi) := \sum_{u \in \omega^+(s)} \varphi(u) - \sum_{u \in \omega^-(s)} \varphi(u),$$

c'est-à-dire le flux net sortant de la source (équivalent au flux net entrant dans le puits). Enfin, on dit qu'un arc u est *saturé* si $\varphi(u) = c(u)$.

Exemple Voici un exemple de réseau de transport :

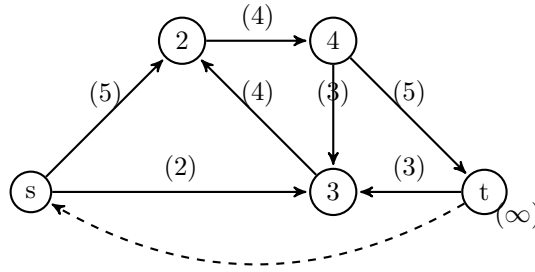


FIGURE 1.4 – Réseau de Transport

Définition (Flot maximal) . Un *flot maximal* d'un réseau est un flot compatible qui maximise la valeur $v(\varphi)$ parmi tous les flots compatibles du réseau.

1.3.2 Théorème de Coupure minimale

Pour construire un flot maximal dans un réseau, nous avons besoin d'un peu de théorie.

Définition (Coupure et cocycle) . Soit $R = (X, E, c)$ un réseau, où X est l'ensemble des sommets, E l'ensemble des arêtes et $c : E \rightarrow \mathbb{R}^+$ la fonction de capacité.

Une *coupure* ct séparant s de t est un couple $(A, X \setminus A)$ tel que :

$$A \subset E, \quad s \in A, \quad t \notin A.$$

L'ensemble des arcs de la coupure, noté $\text{arcs}(ct)$ est l'ensemble :

$$\text{arcs}(ct) = \omega^+(A) := \{u = (i, j) \in E \mid i \in A, j \in X \setminus A\}.$$

Qui correspond à l'ensemble des arcs qui, si on les enlève, coupe tout les chemins entre s et t .

Définition (Capacité d'une coupure) . La *capacité* d'une coupure ct est la somme des

capacités de ses arêtes :

$$\text{capa}(ct) := \sum_{u \in \text{arcs}(ct)} c(u).$$

Théorème (Flot et coupure minimale) . Pour tout flot compatible φ et pour toute coupure ct d'un réseau R , on a :

$$v(\varphi) \leq \text{capa}(ct)$$

1.4 Flot maximal : principe de Ford-Fulkerson

L'objectif de l'algorithme de Ford-Fulkerson est de construire de manière itérative un flot maximal. Pour cela, on part du flot nul dans le réseau. Et petit-à petit, on cherche des chemins non saturés de la source s vers le puit t pour lesquels on peut augmenter leur flot. Une fois que l'on n'est plus capable de trouver de chemin, alors l'algorithme se termine. La définition ci-dessous détaille le processus de construction d'un tel chemin.

1.4.1 Recherche de Flot Maximal

Définition (Principe de Ford-Fulkerson) . Soit $R = (X, E, c)$ un réseau et φ un flow sur R . L'algorithme de construction d'un chemin non saturé est :

1. **Initialisation** : On marque s
2. **Itération** : Pour tout sommet marqué $x, y \in X$ peut être marqué si :
 - y n'est pas encore marqué
 - $\begin{cases} \exists u = (x, y) \in E, \varphi(u) < c(u) & \text{(marquage direct)} \\ \exists u = (y, x) \in E, \varphi(u) < c(u) & \text{(marquage indirect)} \end{cases}$

L'algorithme se termine si on arrive à marquer le puit t . Dans ce cas, la chaîne produite est alors appelée *chaîne augmentante*, notée ch . On note ch^+ les sommets de ch marqués directement et ch^- les sommets marqués indirectement.

Propriété (Calcul d'un nouveau flot) . Soit $R = (X, E, c)$ un réseau et φ un flow sur R . Soit ch une chaîne augmentante de R . Il est alors possible d'augmenter la valeur du flot φ de $k \in \mathbb{R}$ en ajoutant à chaque arcs le flot défini par :

$$k = \min \left(\min_{u \in ch^+} \{c(u) - \varphi(u)\}, \min_{u \in ch^-} \{\varphi(u)\} \right)$$

Le nouveau flot est obtenu en augmentant de k le flow des arcs de ch^+ et en diminuant le flot de k des arcs de ch^- .

Théorème (Coupure Minimale - Flot Maximal) . Soit $R = (X, E, c)$ un réseau. Soit F l'ensemble des flots possibles sur R et K l'ensemble des coupures possibles de R . Alors la valeur du flot maximal est égale à la capacité de la coupure minimale.

$$\text{i.e. } \max_{\varphi \in F} \{v(\varphi)\} = \min_{ct \in K} \{\text{capa}(ct)\}$$

Ce théorème nous permet donc de trouver directement la valeur du flot maximal que l'on peut faire passer dans notre réseau sans avoir à en construire un.

Remarque La dernière itération de l'algorithme de Ford-Fulkerson nous donne une coupure minimale. Les sommets marqués correspondent aux sommets de l'ensemble A de la coupure $ct = (AX/A)$.

1.4.2 Recherche de flot maximal de coût minimal

Rajoutons maintenant une contrainte dans la recherche de notre flot maximal. On souhaite trouver un flot maximal dans un réseau pour lequel le coût du flot sera le plus faible. On cherche donc à faire passer le plus de marchandise dans notre réseau par les routes qui "coûtent le moins cher".

Nous devons donc définir un nouveau type de réseau intégrant cette notion de coût.

Définition (Réseau à coût) . Soit $R = (X, E, c)$ un réseau de noeuds E , d'arcs E et de fonction de capacité c . On appelle *réseau à coût* le réseau R auquel on a rajouté une fonction de coût γ , qui à chaque arête lui associe un coût :

$$\gamma : \begin{cases} E \longrightarrow \mathbb{R} \\ (x, y) \longmapsto \gamma(x, y) \end{cases}$$

Exemple On représentera donc un réseau à coût comme un réseau normal mais en rajoutant une valeur numérique sur chaque arête correspondant à son coût :

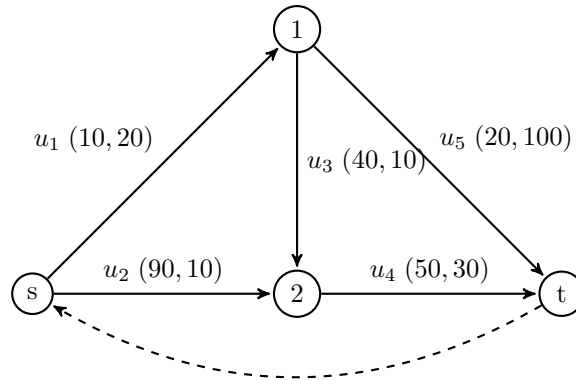


FIGURE 1.5 – Réseau de transport à coût

Remarque Chercher un flot maximal de coût minimal dans R revient donc à construire un flot via l'algorithme de Ford-Fulkerson en choisissant une chaîne augmentante de coût minimal à chaque itération.

Pour trouver cette chaîne augmentante de coût minimal, nous devons définir le graphe résiduel d'un réseau.

Définition (Graphe Résiduel) . Soit $R = (X, E, c, \gamma)$ un réseau et φ un flot sur R , on définit le *graphe résiduel* $G_R(\varphi) = (X, U_\varphi, r_\varphi)$ associé à R et φ comme le graphe tel que pour tout arc $(x, y) \in E$, on ait :

- un *arc direct* $(x, y) \in U_\varphi$ si $\varphi(x, y) < c(x, y)$ avec $r_\varphi = c(x, y) - \varphi(x, y)$
- un *arc indirect* $(y, x) \in U_\varphi$ si $\varphi(y, x) > 0$ avec $r_\varphi = \varphi(y, x)$.

Chapitre 2

Programmation Linéaire

Contents

2.1	Généralités	11
2.1.1	Définition et exemples	11
2.1.2	Solution Géométrique	14
2.2	Algorithme du Simplexe et Dualité	15
2.2.1	Principe du simplexe	15
2.2.2	Dualité	18

Intéressons nous maintenant aux problèmes de programmation linéaire. Même si le nom peut prêter à confusion, il s'agit en réalité de problèmes d'optimisation d'une *fonction objectif* en respectant certaines *contraintes*. Le caractère linéaire de tels problèmes vient du fait que nous ne traiterons que des fonctions objectifs et des contraintes linéaires.

Ces problèmes de programmation linéaires peuvent être résolus en temps polynomial, notamment grâce à l'algorithme du simple. Les problèmes de flots maximaux, vus précédemment, peuvent s'exprimer comme des problèmes de programmation linéaire.

2.1 Généralités

2.1.1 Définition et exemples

Définition (Programme linéaire) . Un *programme linéaire* ou *problème de programmation linéaire* est un problème d'optimisation de $n \in \mathbb{N}$ variables et $m \in \mathbb{N}$ contraintes de la forme :

$$\begin{aligned} \text{minimize} \quad & z = \sum_{j=1}^n c_j x_j \\ \text{subject to} \quad & \sum_{j=1}^n a_{ij} x_j \preceq b_i \quad \forall i \in \llbracket 1, m \rrbracket, \\ & x \succeq 0 \quad \forall j \in \llbracket 1, n \rrbracket \end{aligned}$$

où $\preceq \in \{\leq, \geq, =\}$. Un programme linéaire est donc un simple problème d'optimisation sur un polyèdre convexe. Dans le cas défini plus haut, on dit que le programme est sous forme générale. Différentes formes d'un même problème existent.

Exemple Voyons dès à présent un exemple de programme linéaire.

Un barman possède 30 litres de jus d'orange, 24 litres de pamplemousse et 18 litres de jus de framboise. Il peut préparer 2 types de cocktails :

- *Le "sunrise", vendu 80 euros le baril de 10 litres composé de 5 litres de jus d'orange, de 2 litres de jus de pamplemousse, d'un litre de jus de framboise et de 2 litres d'eau.*
- *Le "swing", vendu 60 euros le baril et composé de 3 litres de jus de pamplemousse, 3 litres de jus d'orange, 3 litres de jus de framboise et d'un litre d'eau.*

Il souhaite préparer la plus importante quantité des deux cocktails pour maximiser son profit.

Pour résoudre ce problème, nous pouvons construire un programme linéaire de deux variables x et y représentant chacune le nombre de litres d'un des deux cocktails à produire. La fonction objectif, décrivant le prix de revente de la marchandise est donc :

$$z = 80x + 60y$$

Et nous avons les contraintes suivantes :

$$\begin{aligned} 5x + 3y &\leq 30 && \text{(stock de jus d'orange)} \\ 2x + 3y &\leq 24 && \text{(stock de jus de pamplemousse)} \\ x + 3y &\leq 18 && \text{(stock de jus de framboise)} \\ x, z &\geq 0 \end{aligned}$$

On peut donc formuler le problème suivant :

$$\begin{aligned} \text{maximize} \quad & z = 80x + 60y \\ \text{subject to} \quad & 5x + 3y \leq 30, \\ & 2x + 3y \leq 24, \\ & x + 3y \leq 18, \\ & x, y \geq 0 \end{aligned}$$

Proposition Comme dit dans la définition d'un programme linéaire, il peut exister différentes formes d'un même problème en fonction de la méthode de résolution utilisée. Le caractère linéaire de ces programmes nous permet de passer facilement d'une forme à une autre. Nous définissons la forme canonique d'un programme linéaire comme suit :

$$\begin{aligned} \text{maximize} \quad & z = \sum_{j=1}^n c_j x_j \\ \text{subject to} \quad & \sum_{j=1}^n a_{ij} x_j \leq b_i, \quad \forall i \in \llbracket 1, m \rrbracket, \\ & x_j \geq 0, \quad \forall j \in \llbracket 1, n \rrbracket \end{aligned}$$

FIGURE 2.1 – Formulation canonique d'un programme linéaire

$$\begin{aligned}
&\text{maximize} && z = \sum_{j=1}^n c_j x_j \\
&\text{subject to} && \sum_{j=1}^n a_{ij} x_j = b_i, \quad \forall i \in \llbracket 1, m \rrbracket, \\
&&& x_j \succeq 0, \quad \forall j \in \llbracket 1, n \rrbracket
\end{aligned}$$

Nous pouvons ensuite passer à la formulation dite « standard » où les contraintes d'ordre sont remplacées par des contraintes d'égalités. Il faut donc rajouter un certain nombre de variables à notre programme.

FIGURE 2.2 – Formulation standard d'un programme linéaire de maximisation

Enfin, nous avons la forme matricielle, qui nous permettra d'utiliser des algorithmes itératifs pour résoudre ce genre de problèmes :

$$\begin{aligned}
&\text{maximize} && z = CX \\
&\text{subject to} && AX = B, \\
&&& X \geq 0_{\mathbb{R}^n}
\end{aligned}$$

FIGURE 2.3 – Formulation matricielle d'un programme linéaire de maximisation

où $C \in \mathbb{R}^n$, $A \in \mathcal{M}_{m,n}(\mathbb{R})$ et $B \in \mathbb{R}^m$.

Exemple Pour le problème de notre barman, nous pouvons ainsi représenter les différentes formes de notre problème :

$$\begin{aligned}
&\text{maximize} && z = 80x + 60y \\
&\text{subject to} && 5x + 3y \leq 30, \\
&&& 2x + 3y \leq 24, \\
&&& x + 3y \leq 18, \\
&&& x, y \geq 0
\end{aligned}$$

FIGURE 2.4 – Forme canonique du problème du barman

$$\begin{aligned}
&\text{maximize} && z = 80x + 60y \\
&\text{subject to} && 5x + 3y + e_1 = 30, \\
&&& 2x + 3y + e_2 = 24, \\
&&& x + 3y + e_3 = 18, \\
&&& x, y, e_1, e_2, e_3 \geq 0
\end{aligned}$$

FIGURE 2.5 – Forme standard du problème du barman

Et enfin :

$$\begin{array}{ll} \text{maximize} & z = \underbrace{(80, 60, 0, 0, 0)}_C \times \underbrace{\begin{pmatrix} x \\ y \\ e_1 \\ e_2 \\ e_3 \end{pmatrix}}_X \\ \text{subject to} & \underbrace{\begin{pmatrix} 5 & 3 & 1 & 0 & 0 \\ 2 & 3 & 0 & 1 & 0 \\ 1 & 3 & 0 & 0 & 1 \end{pmatrix}}_A \times \underbrace{\begin{pmatrix} x \\ y \\ e_1 \\ e_2 \\ e_3 \end{pmatrix}}_X = \underbrace{\begin{pmatrix} 30 \\ 24 \\ 18 \end{pmatrix}}_B \end{array}$$

Maintenant que nous avons défini formellement un programme linéaire, passons à l'étude de ses solutions.

Définition (Solutions d'un programme linéaire) . Soit P un programme linéaire sous forme matricielle standard. Le vecteur $X = (x_1, \dots, x_n)$ est :

- *une solution de P* si $A^t X = B$
- *une solution possible de P* si $A^t X = B$ et $X \geq 0_{\mathbb{R}^n}$
- *une solution optimale de P* si c'est une solution possible et qu'elle maximise z parmi l'ensemble des solutions possibles.

Proposition Comme énoncé en introduction, on peut représenter un problème de maximisation de flot dans un réseau par un programme linéaire. Soit $R = (X, E, c)$ un réseau tel que $|E| = n$, avec $s, t \in X$ la source et le puits. On note $c(x, y)$ la capacité de l'arête entre x et y dans le réseau. On cherche une application

$$\varphi : E \longrightarrow \mathbb{R}, \quad (x, y) \longmapsto f(x, y)$$

qui maximise le flot entre la source s et le puits t .

Plus formellement, si l'on ordonne les arêtes E sous la forme $E = \{e_1, \dots, e_n\}$ et que l'on note $\varphi(E) = (f_1, \dots, f_n) \in \mathbb{R}^n$, alors on cherche à maximiser

$$z = \sum_{(x, y) \in v^-(s)} \varphi(x, y)$$

sous les contraintes :

$$0 \leq f(x, y) \leq c(x, y), \quad \forall (x, y) \in E$$

ainsi que les contraintes de conservation du flot pour tout sommet $v \in X \setminus \{s, t\}$:

$$\sum_{(u, v) \in v^-(v)} f(u, v) = \sum_{(v, w) \in v^+(v)} f(v, w).$$

2.1.2 Solution Géométrique

La recherche d'une solution d'un programme linéaire revient à trouver une droite de la forme $z = \sum_{i=1}^n x_i c_i$ pour laquelle z est maximal tout en respectant les contraintes du programme. En petite dimension (2 variables ou moins) il est possible de tracer ce problème et de le résoudre géométriquement. Détaillons cela autour d'un exemple.

Exemple (Solution Géométrique) Reprenons le problème de notre barman énoncé comme suit :

$$\begin{aligned} \text{maximize} \quad & z = 80x + 60y \\ \text{subject to} \quad & 5x + 3y \leq 30, \\ & 2x + 3y \leq 24, \\ & x + 3y \leq 18, \\ & x, y \geq 0 \end{aligned}$$

On peut tracer les courbes des contraintes pour identifier la zone contenant les solutions possibles :

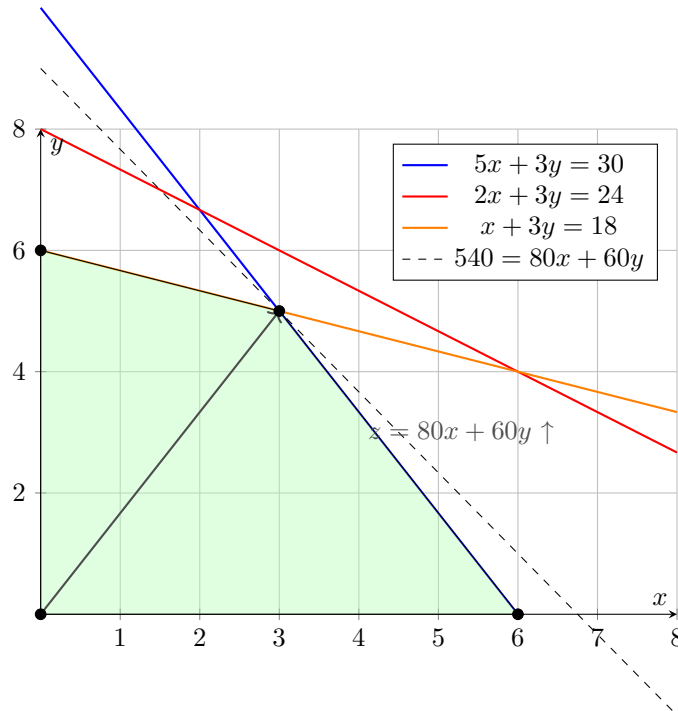


FIGURE 2.6 – Espace des solutions réalisables du programme linéaire
Le barman peut donc produire 80 litres de "sunrise" et 3 litres de "swing" pour un profit total de 540 euros.

Remarque En pratique, il sera très rare de pouvoir résoudre un tel problème graphiquement car le nombre de variable sera bien supérieur à 2.

2.2 Algorithme du Simplexe et Dualité

2.2.1 Principe du simplexe

Comme dit précédemment, le caractère linéaire des contraintes d'un programme linéaire nous permettent d'en déduire que l'espace des solutions possible est un polyèdre. D'où le théorème suivant.

Théorème (Solutions Possibles) . L'ensemble S des solutions possibles d'un programme linéaire est convexe et est un polyèdre non borné ou un polytope. Si S est non vide et borné, alors il existe au moins un vecteur de S pour lequel le maximum de la fonction objectif est atteint.

Remarque Ce théorème nous permet de distinguer plusieurs cas pour l'ensemble des solutions :

- Le programme linéaire a au moins une solution :
 - soit elle est unique
 - soit il en existe une infinité (i.e le maximum est atteint en plusieurs vecteurs) on parle alors de cas dégénéré.
- Le programme linéaire n'a pas de solution, lorsque les contraintes ne sont pas compatibles, l'ensemble des solutions est donc vide.
- Le programme linéaire n'a pas d'optimum (i.e l'ensemble des solutions n'est pas borné).

Proposition (Résolution d'un programme linéaire par le simplexe) Nous avons précédemment résolu nos programmes linéaires graphiquement. Nous allons voir que nous pouvons aussi les résoudre *algébriquement*, en utilisant l'*algorithme du simplexe*. Soit le programme suivant :

$$\begin{aligned}
 \text{maximize} \quad & z = x + 2y \\
 \text{subject to} \quad & 4x + 2y \leq 300, \\
 & 5x \leq 310, \\
 & 2x + 4y \leq 200, \\
 & 3y \leq 100, \\
 & x, y \geq 0
 \end{aligned}$$

On passe ce programme sous *forme standard* en introduisant des variables d'écart $e_1, e_2, e_3, e_4 \geq 0$:

$$\begin{aligned}
 \text{maximize} \quad & z = x + 2y \\
 \text{subject to} \quad & 4x + 2y + e_1 = 300, \\
 & 5x + e_2 = 310, \\
 & 2x + 4y + e_3 = 200, \\
 & 3y + e_4 = 100, \\
 & x, y, e_1, e_2, e_3, e_4 \geq 0
 \end{aligned}$$

Le vecteur $(x, y, e_1, e_2, e_3, e_4) = (0, 0, 300, 310, 200, 100)$ est une *solution de base réalisable* : toutes les contraintes sont vérifiées et les variables sont positives. L'idée du simplexe est de :

1. Choisir une *base initiale* (ici, les variables d'écart e_i).
2. Exprimer les variables hors base (x, y) en fonction des variables de base.
3. Augmenter la valeur de la fonction objectif z en remplaçant une variable de base par une variable hors base qui améliore z .
4. Répéter jusqu'à ce qu'aucune amélioration ne soit possible : la solution est alors optimale.

Formellement, le simplexe cherche à écrire le système sous la forme :

$$\begin{cases} z = C + \sum_i c_i x_i \\ x_B = b - Ax_N \end{cases} \quad \text{où } x_B \geq 0, x_N \geq 0$$

À chaque itération :

- une variable hors base entre dans la base (celle qui augmente le plus z),
- une variable de base quitte la base (celle qui devient nulle en premier).

Lorsque tous les coefficients $c_i \leq 0$, aucune amélioration n'est possible et la solution actuelle est *optimale* : la valeur optimale est alors $z = C$.

Exemple (Résolution algébrique du programme par le simplexe) Considérons le programme linéaire suivant :

$$\begin{aligned} \text{maximize} \quad & z = x + 2y \\ \text{subject to} \quad & 4x + 2y \leq 300, \\ & 5x \leq 310, \\ & 2x + 4y \leq 200, \\ & 3y \leq 100, \\ & x, y \geq 0 \end{aligned}$$

1. **Mise sous forme standard** : On introduit une variable d'écart pour chaque contrainte :

$$\begin{cases} 4x + 2y + e_1 = 300 \\ 5x + e_2 = 310 \\ 2x + 4y + e_3 = 200 \\ 3y + e_4 = 100 \end{cases} \quad \text{avec } e_i \geq 0.$$

La fonction objectif s'écrit :

$$z = x + 2y.$$

Une solution de base réalisable évidente est obtenue en prenant $x = y = 0$, ce qui donne :

$$(e_1, e_2, e_3, e_4) = (300, 310, 200, 100), \quad z = 0.$$

C'est notre solution de base initiale.

2. **Recherche d'une direction d'amélioration** : Comme $z = x + 2y$, augmenter y est plus rentable que x (le coefficient 2 est supérieur à 1). On va donc chercher à **augmenter** y tant que c'est possible sans violer les contraintes. On part du système :

$$\begin{cases} e_1 = 300 - 2y - 4x \\ e_2 = 310 - 5x \\ e_3 = 200 - 2x - 4y \\ e_4 = 100 - 3y \end{cases}$$

Comme on garde $x = 0$ au départ, ces équations deviennent :

$$\begin{cases} e_1 = 300 - 2y \\ e_2 = 310 \\ e_3 = 200 - 4y \\ e_4 = 100 - 3y \end{cases}$$

Pour rester faisable, il faut $e_i \geq 0$. On obtient donc :

$$y \leq 150, \quad y \leq 50, \quad y \leq 25, \quad y \leq 33,3.$$

La contrainte la plus restrictive est $y \leq 25$ (provenant de $e_3 = 200 - 4y$). On choisit donc $y = 25$ et $e_3 = 0$. Cela définit une nouvelle base : $\{e_1, e_2, e_4, y\}$.

3. **Amélioration suivante :** Pour cette valeur $y = 25$, les variables d'écart valent :

$$e_1 = 300 - 2(25) = 250, \quad e_2 = 310, \quad e_3 = 0, \quad e_4 = 100 - 3(25) = 25.$$

La fonction objectif vaut alors :

$$z = x + 2y = 2 \times 25 = 50.$$

On peut encore améliorer z en augmentant x , tant que toutes les contraintes restent valides. En remplaçant $y = 25$, on obtient :

$$\begin{cases} e_1 = 300 - 2(25) - 4x = 250 - 4x \\ e_2 = 310 - 5x \\ e_3 = 200 - 2x - 4(25) = 100 - 2x \\ e_4 = 100 - 3(25) = 25 \end{cases}$$

Les contraintes donnent :

$$x \leq 62,5, \quad x \leq 62, \quad x \leq 50.$$

Le plus restrictif est $x \leq 50$ (provenant de $e_3 \geq 0$). On prend donc $x = 50$ et $e_3 = 0$.

4. **Calcul de la solution optimale :** On obtient :

$$\begin{cases} x = 50, \quad y = 25, \\ e_1 = 300 - 4(50) - 2(25) = 100, \\ e_2 = 310 - 5(50) = 60, \\ e_3 = 0, \\ e_4 = 100 - 3(25) = 25. \end{cases}$$

La fonction objectif vaut :

$$z = x + 2y = 50 + 2 \times 25 = 100.$$

Si on essaye d'augmenter y ou x davantage, on viole immédiatement une contrainte. La solution est donc **optimale**.

5. **Conclusion :** Le programme atteint son maximum en :

$$\boxed{x^* = 50, \quad y^* = 25, \quad z_{\max} = 100.}$$

Cette résolution illustre la logique du simplexe : on part d'une solution réalisable triviale, puis on améliore la fonction objectif pas à pas en déplaçant la base vers les sommets adjacents du polytope faisable, jusqu'à atteindre un sommet où toute augmentation violerait une contrainte.

2.2.2 Dualité

Définition (Programme Primal) . Le *problème primal* (ou primaire) est la forme standard d'un programme linéaire :

$$\begin{aligned} & \text{maximize} && z = {}^t c x \\ & \text{subject to} && Ax \leq b, \\ & && x \geq 0 \end{aligned}$$

où :

- $x \in \mathbb{R}^n$ est le vecteur des variables de décision ;
- $c \in \mathbb{R}^n$ est le vecteur des coefficients de la fonction objectif ;
- $A \in \mathbb{R}^{m \times n}$ est la matrice des coefficients des contraintes ;
- $b \in \mathbb{R}^m$ est le vecteur des bornes du système d'inégalités.

Définition (Problème Dual) . À tout problème primal est associé un *problème dual* défini par :

$$\begin{aligned} & \text{minimize} && w = {}^t b y \\ & \text{subject to} && {}^t A y \geq c, \\ & && y \geq 0 \end{aligned}$$

où :

- $y \in \mathbb{R}^m$ est le vecteur des variables duales ;
- chaque contrainte du primal correspond à une variable du dual ;
- chaque variable du primal correspond à une contrainte du dual.

Remarque (Interprétation) Le problème *primal* cherche à *maximiser un profit* ou une valeur objective en respectant des contraintes de ressources. Le problème *dual* cherche à *minimiser le coût* de ces ressources tout en garantissant que la valeur de chaque ressource couvre la contribution de chaque variable du primal. Autrement dit :

Le primal cherche ce qu'on peut obtenir au maximum avec des ressources limitées, tandis que le dual cherche le coût minimal que doivent avoir ces ressources pour que le système reste équilibré.

Exemple En reprenant l'exemple de notre barman, le programme primal cherche à maximiser le profit du barman en fixant le prix des barrils c_j de cocktails j qu'il produit en utilisant au plus b_i litres de jus i . Les variables x_j représentent alors le nombre de barrils de cocktail j .

Ici, le programme dual cherche à minimiser le profit total obtenu grâce aux stocks b_i de chaque jus i si le barman vend chaque cocktail j pour un prix minimal de c_j . Les variables y_i représentent donc le profit que le barman réalise par litre de jus i .

Voici les expressions mathématiques de ces deux programmes :

$$\begin{array}{ll}
\text{maximize} & z = 80x + 60y \\
\text{subject to} & 5x + 3y \leq 30, \\
& 2x + 3y \leq 24, \\
& x + 3y \leq 18, \\
& x, y \geq 0
\end{array}$$

FIGURE 2.7 – Programme primal du barman

$$\begin{array}{ll}
\text{minimize} & w = 20y_1 + 24y_2 + 18y_3 \\
\text{subject to} & 5y_1 + 2y_2 + y_3 \geq 80, \\
& 3y_1 + 3y_2 + 3y_3 \geq 60, \\
& y_1, y_2, y_3 \geq 0
\end{array}$$

FIGURE 2.8 – Programme dual du barman

La solution optimale du primal est $(x_1, x_2) = (3, 5)$ pour un profit maximal de 540. En revanche la solution du dual est $(y_1, y_2, y_3) = (15, 0, 5)$ pour un profit de 540 aussi.

Il existe un lien direct entre les solutions du dual et celles du primal. Elles sont décrites par les théorèmes de dualité.

Théorème (Dualité Faible) . La solution optimale du programme primal est inférieure ou égale à celle du dual.

Théorème (Dualité Forte) . Nous avons les deux implications suivantes :

1. Si l'un des deux programmes (primal ou dual) admet une solution optimale finie, alors l'autre aussi. De plus, la valeur optimale pour des deux programmes est la même.
2. Si l'ensemble des solutions de l'un des deux programmes est non borné, alors celui de l'autre est vide.

